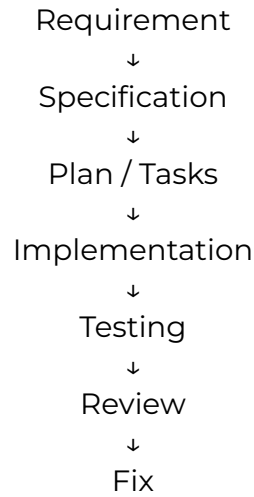


ATL / TL Assignment

Ask

You must use Cursor / Kiro / VS Code with Spec driven development with the below workflow



Do not start by asking AI:

"Build the complete application."

Generic Artefacts

Regardless of the IDE you use, maintain some basic hygiene steering files. They must include at the very least

- Java Springboot Guidelines
- Testing guidelines
- API Standards
- Documentation Skills
- RAG / Vector Store guidelines - chunking convention, embedding model choice, retrieval-tuning defaultsX
- Commands to review code, spec, and generate tests
- A command to review AI assistant output specifically for hallucination / ungrounded answers

If you use cursor, the structure might look something like below

rules/

java-springboot.md

```
testing.md
api-standards.md
rag-vector-store.md
skills/
  documentation/
commands/
  review-code.md
  review-spec.md
  generate-tests.md
  review-rag-output.md
```

The exact structure can vary depending on the IDE you use. The important thing is to understand and demonstrate reusable AI instructions across the project context.

Spec Artefacts

Create specifications before implementation. Once you do, an example structure looks like

```
spec/
├── requirements.md
├── architecture.md
├── data-model.md
├── api-contract.md
├── state-machine.md
├── rag-ingestion.md
├── rag-api-contract.md
├── evaluation-strategy.md
├── ui-flow.md
└── test-strategy.md
```

Prompt History

Any prompt you give must be saved to a file. Extensions like SpecStory allow you to do so for Cursor and VSCode out of the box. If you are using Kiro, create a small skill that records every prompt to a file.

Repository should contain:

```
.specstory/
  history/
and:
docs/
  prompt-history.md
```

Important

You must identify at least meaningful mistakes or incorrect suggestions by the AI - this includes wrong code AND ungrounded or hallucinated assistant answers.

This demonstrates that you are using AI as an engineering assistant, not blindly accepting AI output.

Token Optimisation

Use plugins like Graphify, Caveman, Codebase-memory MCP to optimize your token usage. Additionally, use prompt caching for the static portions of your assistant's system prompt (instructions, guardrail text) so repeated questions don't re-pay for unchanged context.

Exercise

Build an AI-Powered Support Ticket Management System using:

- Java 21
- Spring Boot
- Spring AI
- PostgreSQL/H2
- An Embedding Model
- A Vector Store (e.g. PGVector or Chroma)
- REST API
- React/Next.js or equivalent frontend
- Cursor / GitHub Copilot / Kiro

The application is important, but the main assessment is how you build it using AI - and how you design and reason about the AI assistant specifically.

The main goal is to enable the team to be able to understand

- How to analyse requirements and create specifications for a system that includes an AI-native feature from the start, not bolted on afterward
- How to use spec-driven development for both conventional CRUD features and a RAG pipeline
- How to manage AI context and validate AI-generated code AND AI-generated answers (grounding, hallucination)
- How you test and debug both deterministic logic (the ticket state machine) and probabilistic AI output (retrieval quality)

Application Requirements

Build a Support Ticket Management System with the following features:

- Create a ticket.
- List tickets.
- View ticket details.

- Update title, description, priority and assignee.
- Add comments.
- Search tickets by keyword.
- Filter tickets by status.
- Persist data in a database.
- Validate input at the backend.
- Display meaningful errors in the UI.
- Provide a natural-language question-answering endpoint over ticket history, grounded strictly in real ticket data.
- Cite the specific ticket(s) used to produce any assistant answer.
- Explicitly indicate when no relevant tickets are found, rather than fabricate an answer.

The following state machine must be enforced by the backend:

OPEN → IN_PROGRESS → RESOLVED → CLOSED

OPEN → CANCELLED

IN_PROGRESS → CANCELLED

Invalid transitions must be rejected.

Example:

CLOSED → OPEN ❌

RESOLVED → OPEN ❌

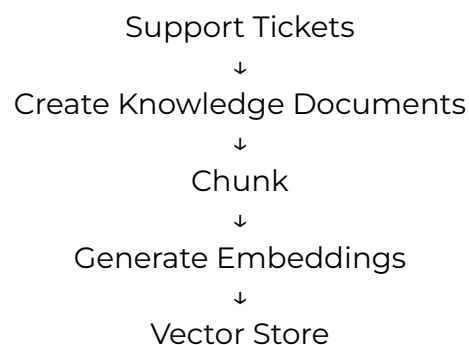
CANCELLED → OPEN ❌

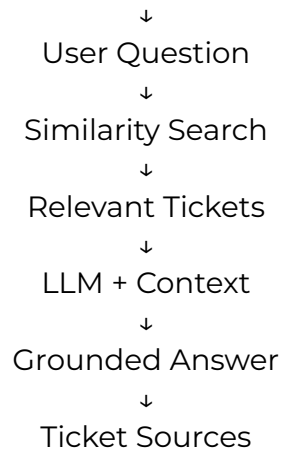
Example Questions the Assistant Should Answer

- “Have we seen payment failures before?”
- “What was the resolution for ticket TKT-1001?”
- “What are the common causes of shipment tracking issues?”
- “Show me similar resolved tickets.”
- “Which high-priority tickets are related to payment?”

Basic RAG Flow

Implement:





RAG & Assistant Requirements

Ingestion

Convert ticket information (description, comments, resolution notes) into searchable knowledge documents. Include metadata:

- ticketId
- status
- priority
- assignee
- category

Re-ingest / refresh embeddings when a ticket is updated or closed - do not let the knowledge base go stale.

API

Create:

POST /api/ai/ask

Request:

```
{
  "question": "What caused previous payment failures?"
}
```

Retrieval Quality

- Documentation around your chunking strategy (paragraph-based vs. fixed-size vs. semantic splitting, for ticket data specifically) - these can be generated based on the brainstorming and spec analysis with the IDE you use
- Top-K and similarity threshold must be configurable, not hardcoded
- Documentation around your embedding model choice (local via Ollama vs. cloud) and the cost/latency/quality tradeoff you considered - these can be generated based on the brainstorming and spec analysis with the IDE you use

Grounding & Guardrails

- The assistant must only answer from retrieved ticket context - it must not fall back on general LLM knowledge for support-specific questions
- If no tickets are relevant, it must say so explicitly rather than produce a fabricated but plausible-sounding answer
- This is deliberately a single retrieval → generate flow, not an autonomous agent. The assistant answers one question with one grounded response - it does not independently decide to take further action (creating tickets, sending notifications, chaining into other tools). That is out of scope.

Core Acceptance Criteria

The solution is complete when:

- Ticket can be created from UI.
- Tickets can be listed.
- Ticket details can be viewed.
- Ticket fields can be updated.
- Assignee can be changed.
- Comments can be added.
- Search works.
- Status filter works.
- Valid status transitions work.
- Invalid status transitions are rejected by backend.
- Data survives application restart.
- Backend validation works.
- UI shows meaningful errors.
- State-machine integration tests pass.
- Ticket data is converted into embeddings and stored in a vector store.
- POST /api/ai/ask returns a grounded, ticket-sourced answer for in-scope questions.
- The response cites the specific ticket ID(s) used to generate it.
- Out-of-scope / no-match questions return an honest “no relevant tickets found” response, not a fabricated answer.
- Chunking strategy and embedding model choice are documented and justified in architecture.md.
- Re-ingestion happens when a ticket is updated - embeddings do not go stale.
- Retrieval parameters (top-K, similarity threshold) are configurable, not hardcoded.
- No secrets are committed.
- At least one meaningful AI mistake - in code or in a RAG answer - was caught and documented during development.